MicroRCA-Agent: Toward Evidence-Grounded Agentic Root Cause Analysis for Microservice Systems

Project Code: [PROJECT CODE]

Ibrahim Umar Malik
Roll No. 22I-2315
FAST-NUCES, Islamabad
[email]

Syed Mustafa Ali
Roll No. 22I-2304
FAST-NUCES, Islamabad
[email]

Fasih Shahid Abbasi
Roll No. 22I-2235
FAST-NUCES, Islamabad
[email]

Hamza Ijaz
Roll No. 22I-2330
FAST-NUCES, Islamabad
[email]

***Abstract*—When a microservice system fails, the user-visible symptom frequently surfaces in a service other than the one that initiated the fault, and operators must reconcile metrics, logs, traces, and call-graph structure to localize the cause. Recent studies apply tool-using large language model (LLM) agents to this task, but report low accuracy, unverified hypotheses, and unbounded investigation cost. This paper presents MicroRCA-Agent, an evaluation-ready framework for agentic root cause analysis (RCA). It contributes typed incident and diagnosis contracts with label blinding, six bounded observability tools, provider-agnostic LLM adapters, statistical and single-call baselines, a ReAct-style investigator with step and tool budgets, and a shared harness that records accuracy together with model calls, tool calls, tokens, and latency. On 101 labelled failures from the Online Boutique and Train-Ticket systems in the Nezha benchmark, reciprocal-rank fusion of metric, log, and trace evidence raises top-1 localization accuracy (AC@1) from 0.178 to 0.307 relative to metrics-only scoring (McNemar exact $p = 0.001$), at a mean cost of 0.009 s per case. A leave-one-signal-out ablation shows that most of this gain is attributable to fusing CPU and memory evidence rather than to logs or traces. A failure analysis of the fused baseline finds that 25 of its 70 top-1 errors rank a direct caller or callee of the true root cause first, and that accuracy collapses on Train-Ticket (AC@1 0.156) and on code-level faults (AC@1 $\leq$0.111). These results quantify where statistical scoring fails and motivate the verification, cost-control, and memory mechanisms of the proposed agent. Accuracy results for the LLM-based methods are not yet available and are not claimed.**

***Index Terms*—microservices, root cause analysis, agentic AI, large language models, observability, ReAct, AIOps**

I. Introduction

Microservice architectures decompose an application into independently deployed services that communicate over the network [1]. A single user request may traverse many such services, so a fault in one component commonly manifests as elevated latency or errors in its callers. The resulting gap between *where a failure is observed* and *where it originates* makes root cause analysis (RCA) one of the most time-consuming activities in incident response [2], [3]. Operators must correlate resource metrics, application logs, and distributed traces, and must reason over the service dependency graph to separate causes from downstream symptoms.

Two families of automated RCA have emerged. Statistical and graph-based methods score services by anomaly magnitude, causal structure, or trace spectra [4]–[10]. They are fast and reproducible, but their scoring rules are fixed and they rarely explain their conclusions. More recently, LLM-based systems have been proposed that reason over incident data and invoke tools [11]–[14]. The OpenRCA benchmark, which we adopt as our base paper, evaluates such agents on 335 failures from three enterprise systems and reports that even the best configuration, an RCA-agent driven by Claude 3.5, solved only 11.34% of failure cases [15].

Research gaps. From the Phase 1 literature review we identify four gaps that motivate this work. *G1 (verification):* agentic RCA systems typically accept the first plausible hypothesis, without checking whether the implicated service is a cause or a propagated symptom [14], [15]. *G2 (cost awareness):* published agents seldom bound or report the number of model calls, tool calls, and tokens consumed per incident, which obscures whether accuracy gains justify their cost [16]. *G3 (incident memory):* each incident is investigated from scratch, even though recurring fault patterns are common in production [12]. *G4 (multimodal grounding):* many methods rely on a single telemetry modality, whereas faults such as exceptions or wrong return values may be visible only in logs or traces [17], [18].

Research questions. This paper addresses the following questions.

- **RQ1 (G4).** Does fusing metric, log, and trace evidence improve service-level localization over metrics-only scoring, and which signals contribute to the improvement?
- **RQ2 (G1).** To what extent are localization errors explained by fault propagation along the call graph,

which a verification step would target?
- **RQ3 (G2).** What are the per-incident costs of the evaluated methods, and can agentic investigation be bounded and accounted for in the same harness?
- **RQ4 (G1–G3).** Does a tool-using agent with verification, budget control, and memory outperform statistical and single-call baselines at acceptable cost? We state RQ4 as a hypothesis with an evaluation protocol; it is not answered in this paper.

Contributions. The contributions of this work are:

1) A reproducible RCA framework with typed contracts, label blinding, compact multimodal incident bundles, and six JSON-schema observability tools shared by all methods [19].
2) A family of comparable methods, namely two statistical baselines (B0), a single-call LLM baseline (B1), and a budgeted ReAct-style investigator (P0) with interfaces for verification and memory, evaluated by one harness that records quality and cost.
3) An empirical characterization of statistical RCA on 101 Nezha incidents, including significance testing, a signal ablation, and a categorized failure analysis that quantifies the symptom-propagation errors targeted by G1.

II. Related Work

Statistical and graph-based RCA. Early microservice RCA methods construct a dependency or anomaly graph and rank services with random walks or PageRank-style propagation. CloudRanger builds an impact graph from metrics using causal discovery [5]; MicroRCA attributes anomalies on an attributed service graph [4]; and MicroCause combines causal inference with temporal propagation to localize failures among metrics [6]. CIRCA formulates RCA as intervention recognition on a causal Bayesian network [9], while BARO detects multivariate change points and ranks root causes robustly [10]. Trace-centric methods such as MicroRank [7] and TraceRCA [8] exploit span structure and spectrum analysis, and Groot constructs event graphs for industrial-scale RCA [20]. Survey work catalogues these approaches and notes their reliance on hand-crafted scoring rules and single data sources [2].

Multimodal RCA. Nezha mines interpretable patterns across metrics, logs, and traces and localizes root causes at code and resource granularity; it also released the fault-injection dataset used in this study [17]. Eadro jointly learns anomaly detection and localization from multi-source data [18]. Benchmarks such as RCAEval [21], together with the Train-Ticket [3], DeathStarBench [22], and Online Boutique [23] systems, enable reproducible comparison. Our statistical baselines borrow the robust-deviation scoring used by such detectors [24] and fuse modalities with reciprocal-rank fusion [25].

LLMs and agents for incident management. Ahmed et al. showed that fine-tuned LLMs can recommend root causes and mitigations for cloud incidents [11]. RCACopilot retrieves historical incidents and diagnostic outputs to predict root-cause categories [12], and RCAgent equips an autonomous agent with tools over cloud data [13]. Roy et al. evaluated ReAct agents on production incidents and reported both promise and failure modes such as hallucinated evidence [14]. These systems build on general agent techniques: chain-of-thought prompting [26], interleaved reasoning and acting (ReAct) [27], tool use [28], self-consistency [29], iterative self-feedback [30], [31], and memory-augmented agents [32]; a broader survey is given in [33]. Cost-aware LLM usage has been studied in FrugalGPT [16], but it is rarely reported in RCA evaluations.

Positioning against the base paper. OpenRCA [15] defines RCA as identifying the originating component, time, and reason of a failure from raw telemetry, and provides an RCA-agent that writes and executes analysis code. Its low solve rate establishes that unconstrained agentic RCA remains difficult. MicroRCA-Agent differs in three respects. First, it exposes a small set of typed, bounded tools rather than open-ended code execution, which constrains the action space and makes tool usage auditable. Second, it evaluates every method, statistical or agentic, through one harness that records cost alongside accuracy (G2). Third, it reserves explicit interfaces for hypothesis verification (G1) and incident memory (G3), which OpenRCA's agent does not provide. Because OpenRCA uses different systems, labels, and a stricter success criterion, its numbers are not directly comparable with ours (Section VI).

III. Problem Formulation

Let a microservice system have a service set $S = \{s_1, \ldots, s_n\}$ and a directed call graph $G = (S, E)$, where $(s_i, s_j) \in E$ indicates that s_i calls s_j. An *incident* c is reported at alert time t_a and is associated with telemetry $\mathcal{T}_c = (M_c, L_c, R_c)$: per-service metric series M_c, log statistics L_c, and trace statistics R_c, each covering a pre-alert reference window and a post-alert window. Each incident carries a ground-truth label $y_c = (s_c^*, f_c^*)$, where $s_c^* \in S$ is the root-cause service and $f_c^* \in F$ is a fault type from the set $F = \{\texttt{cpu_contention}, \texttt{cpu_consumed}, \texttt{network_delay}, \texttt{exception}, \texttt{return}\}$.

Input. A method receives the *blinded* incident $\tilde{c} = (\mathrm{id}, \mathrm{system}, t_a, S, E)$ with y_c removed, and may query $\mathcal{T}_c$ only through the tool interface.

Output. A method returns a diagnosis $D_c = \langle (s_{(1)}, f_{(1)}, p_{(1)}, e_{(1)}), \ldots, (s_{(k)}, f_{(k)}, p_{(k)}, e_{(k)}) \rangle$, an ordered list of candidate services with a predicted fault type, a confidence $p \in [0, 1]$, and textual evidence e, together with a summary. The task is service-level localization: rank s_c^* as high as possible, and predict f_c^* for the top candidate.

Cost. For each incident the harness records $u_c = (\#\text{LLM calls}, \#\text{tool calls}, \#\text{tokens}, \text{latency})$.

Hypotheses. Corresponding to RQ1–RQ4 we test or specify: *H1*, multimodal fusion improves AC@1 over metrics-only scoring; *H2*, a substantial fraction of top-1 errors place a call-graph neighbour of s_c^* first; *H3*, the investigator's cost can be bounded by step and tool budgets and recorded per incident; and *H4*, the full agent improves AC@1 over B0 and B1 at a reported cost. H1 and H2 are tested in Section VII; H3 is verified at the pipeline level; H4 remains open.

IV. Proposed System

Fig. 1 shows the architecture of MicroRCA-Agent. All methods read from the same case store and return the same diagnosis contract to the evaluation harness, which guarantees that differences in results stem from the methods rather than from data handling.

A. Data pipeline and contracts

A preprocessing script converts the raw Nezha data into one compressed JSON bundle per labelled incident. Each bundle retains per-service metric series from 30 min before to 5 min after the alert; per-service log volume, error-like line rate, and log templates absent from fault-free logs; per-service span rate and p50/p90 span latency for both the fault window and fault-free traffic; the call graph; and the label. Log templates are obtained with a lightweight masking of identifiers and numbers, a simplified form of template mining [34]. The loader validates each bundle against Pydantic contracts (`IncidentCase`, `Diagnosis`, `RankedCause`, `AgentState`, `Usage`, `EvalRecord`); fault types are a closed enumeration and confidences are constrained to $[0, 1]$. A `blind()` operation returns a copy of the incident without its label, and only this copy is passed to a method.

B. Telemetry tools

Table I lists the six tools. Each reads only the current incident's telemetry, returns a small JSON object that fits in an LLM context, and is described by a JSON schema compatible with OpenAI- and Anthropic-style function calling. The same functions serve all methods: B0 calls the scoring functions directly, B1 receives their outputs precomputed in a single prompt, and P0 invokes them on demand. Metric anomalies are scored with a robust z-score of the maximum post-alert deviation from the pre-alert median,

$$z_{s,m} = \frac{\max_{t \geq t_a} |x_{s,m}(t) - \mathrm{med}(x_{s,m}^{\mathrm{pre}})|}{\sigma_{s,m}}, \qquad (1)$$

where $\sigma_{s,m} = \max(\mathrm{IQR}/1.349,\ \mathrm{std},\ 0.05|\mathrm{med}|,\ 10^{-3})$ is floored to prevent flat series from producing unbounded scores [24], and z is capped at 1000. The log score is $\log(1+\Delta_{\mathrm{err}}) + 0.3\, n_{\mathrm{new}}$, where Δ_{err} is the increase in error-like lines per minute and n_{new} the number of new templates; the trace score is $\max(\log(p90_{\mathrm{fault}}/p90_{\mathrm{normal}}), 0)$.

TABLE I
Observability tools exposed to the agent.

Tool	Returns
`list_services`	Services of the system under diagnosis.
`get_call_graph`	All caller–callee edges, or the callers and callees of one service.
`detect_metric_anomalies`	Top-k (service, metric) pairs ranked by robust z-score (1) after the alert.
`get_metric_summary`	Pre-alert mean and standard deviation and post-alert values for one metric.
`get_log_summary`	Log and error-line rates versus fault-free logs, and new templates.
`get_trace_summary`	Span rate and p50/p90 latency versus fault-free traffic.

C. Methods

B0-stat-metric scores each service by its largest robust z-score over eight key metrics (CPU, memory, network receive and transmit, client and server p90 latency, workload, success rate) and maps the dominant metric to a fault type. **B0-stat-multimodal** ranks services separately on five signals (CPU, memory, server p90 latency, trace latency, and log deviation) and fuses the rankings with reciprocal-rank fusion [25], $\mathrm{RRF}(s) = \sum_j 1/r_j(s)$, where $r_j(s)$ is the rank of s under signal j. **B1-single-call** packs the system description, call graph, the top eight metric anomalies, and log and trace summaries for the five leading suspects into one prompt, and the LLM answers once with a diagnosis in JSON. **P0-react-agent** is the proposed investigator described next.

D. Investigator agent and state

P0 follows the ReAct pattern [27]. Its state (`AgentState`) holds the blinded incident, the step counter, the message history, the tool observations, the current hypotheses, and a `Usage` record. The loop proceeds as follows. (i) The system prompt instructs the model to act as a site reliability engineer, to inspect the strongest anomalies, and to determine whether each is a cause or a downstream symptom; the user message gives the system, alert time, and service list. (ii) While fewer than `max_steps` $= 8$ steps have elapsed, the model is called with tools enabled if fewer than `max_tool_calls` $= 20$ tool calls have been made. (iii) Requested tools are executed and their results, truncated to 4000 characters, are appended as observations. (iv) The loop ends when the model replies without a tool call; if a budget is exhausted, the model is asked for a final answer without tools. (v) A tolerant parser extracts the first JSON object, discards malformed entries, and maps unknown fault types to `unknown`. (vi) If configured, a verifier may revise the diagnosis and a memory module may store it. All LLM calls use temperature 0.

E. Components addressing each gap

The *tool layer* and B0-stat-multimodal address G4 by making every modality available as evidence. The *budget*

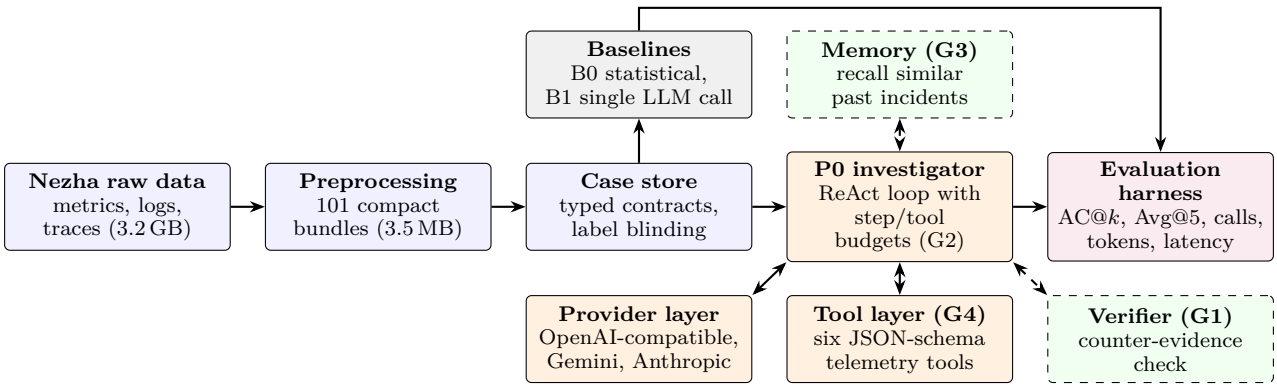


Fig. 1. Architecture of MicroRCA-Agent. The case store passes only label-blinded incidents to every method, and all methods return the same diagnosis contract to the evaluation harness. Solid boxes are implemented in Phase 1; dashed boxes are interfaces whose policies (verification for G1, memory for G3) are planned for Phase 2. Budget control (G2) is enforced inside the investigator loop.

controller (step and tool caps with usage accounting) addresses G2. The *verifier* interface, a function from (incident, diagnosis) to a revised diagnosis, is the insertion point for counter-evidence checks and self-consistency voting [29], [30] that target G1. The *memory* protocol, with `recall` and `store` operations, injects summaries of similar past incidents into the initial context and writes back new diagnoses, following retrieval-based designs [12], [32], to address G3. Table IV in Section VII reports the status of each component.

F. Provider layer

A provider-agnostic `ChatModel` interface supports the OpenAI API and any OpenAI-compatible endpoint (including locally served open-weight models), Google Gemini, and the Anthropic Messages API, using only an HTTP client. A deterministic *scripted* provider follows a fixed investigation script (metric anomalies, then logs and traces of the two leading suspects, then an answer listing the metric suspects). It is not a language model; it exists only to exercise the full pipeline offline.

V. Experimental Setup

Dataset. We use the public fault-injection dataset released with Nezha [17], which contains 101 labelled failures on Online Boutique (OB) [23] and Train-Ticket (TT) [3]. Table II summarizes the cases. The raw data occupy about 3.2 GB; the processed bundles occupy about 3.5 MB. The OB bundles contain 10 services and 26 call edges; the TT bundles contain 46 services and 55 call edges. (The repository documentation lists 47 TT services; the processed bundles expose 46.) Faults are unevenly distributed: network delay is the largest class (30 cases) and CPU-consumed faults occur only in OB.

Splits. All four methods are training-free: B0 has no learned parameters and B1/P0 use pretrained LLMs without fine-tuning. Consequently, all 101 incidents form a single test set and no training or validation split is required; the scoring constants of B0 were fixed in Phase 1 and not tuned per case. RCAEval [21] is designated as a held-out set for Phase 2.

TABLE II
Evaluation cases by system and fault type.

System	CPU-ct	CPU-cs	Net	Exc	Ret	Total
Online Boutique	16	10	16	7	7	56
Train-Ticket	7	0	14	13	11	45
Total	23	10	30	20	18	101

CPU-ct: CPU contention; CPU-cs: CPU consumed; Net: network delay; Exc: exception; Ret: wrong return value.

Baselines and models. B0-stat-metric and B0-stat-multimodal are evaluated in full. B1 and P0 are implemented and executed end-to-end with the scripted provider; runs with hosted or locally served LLMs are configured (e.g., an OpenAI-compatible endpoint at temperature 0) but their results are not yet available.

Implementation. The framework is written in Python ($\geq$3.10) and depends only on NumPy, pandas, Pydantic, and httpx. Each run writes a summary JSON (overall, per system, and per fault type) and a per-case JSONL file. A test suite of seven tests, requiring no API key, covers contracts, tools, every method, the parser, the metrics, and the command-line runner. For this paper we re-ran both B0 baselines from the repository and obtained identical accuracy values to those committed in the repository; only wall-clock latency differs by a few milliseconds.

VI. Evaluation Methodology

Localization metrics. For a set C of incidents and a ranked prediction list $\hat{s}_{c,1}, \hat{s}_{c,2}, \ldots$ per incident,

$$\mathrm{AC@}k = \frac{1}{|C|} \sum_{c \in C} \mathbf{1}\left[s_c^* \in \{\hat{s}_{c,1}, \ldots, \hat{s}_{c,k}\}\right], \tag{2}$$

$$\mathrm{Avg@5} = \frac{1}{5} \sum_{k=1}^{5} \mathrm{AC@}k, \tag{3}$$

TABLE III
SERVICE-LEVEL LOCALIZATION ON 101 INCIDENTS. OB AND TT COLUMNS GIVE AC@1 PER SYSTEM. FT: FAULT-TYPE ACCURACY GIVEN A CORRECT TOP-1 SERVICE.

Method	AC@1	AC@3	AC@5	Avg@5	OB	TT	FT
B0-stat-metric	0.178	0.356	0.416	0.323	0.321	0.000	0.222
B0-stat-multimodal	**0.307**	**0.396**	**0.475**	**0.386**	**0.429**	**0.156**	**0.645**
B1-single-call	LLM results not yet available						
P0-react-agent	LLM results not yet available						

following the conventions of Nezha, RCAEval, and related studies [10], [17], [21]. We report AC@1, AC@3, AC@5, and Avg@5 overall, per system, and per fault type.

Fault-type accuracy. Among incidents whose top-1 service is correct, we report the fraction whose predicted fault type equals f_c^*. This conditional definition separates fault classification from localization.

Cost metrics. Per incident we record LLM calls, tool calls, input and output tokens, and wall-clock latency, and report their means.

Statistical testing. Because B0 variants are evaluated on the same incidents, we compare them with an exact McNemar test on discordant top-k outcomes and report 95% percentile bootstrap confidence intervals (10,000 resamples of incidents, fixed seed) for each AC@k and for the paired difference.

Ablation and failure categorization. We ablate the fused baseline by removing one signal at a time and by using each signal alone, keeping the RRF rule fixed. Each top-1 error of B0-stat-multimodal is assigned to exactly one category: *F1, propagation*, where the top-ranked service is a direct caller or callee of s_c^*; *F2, near miss*, where s_c^* is within the top five but F1 does not apply; and *F3, missed*, where s_c^* is ranked below fifth.

Comparison with the base paper. OpenRCA [15] counts a case as solved only if the predicted root-cause component, occurrence time, and reason all match, on 335 failures from three enterprise systems. Our task predicts only the service and fault type, on different systems and labels. We therefore compare at the level of protocol and design (Section VIII) and do not place OpenRCA's 11.34% solve rate and our AC@1 values on a common scale. A re-execution of OpenRCA's agent on our incidents is planned but not reported here.

VII. RESULTS

Table III reports the main localization results, and Table IV summarizes the status of each research gap.

A. G4: Multimodal evidence (RQ1)

Fusing metric, log, and trace rankings raises AC@1 from 0.178 to 0.307, an absolute gain of 0.129. The bootstrap 95% confidence intervals are $[0.109, 0.257]$ for B0-stat-metric and $[0.218, 0.396]$ for B0-stat-multimodal, and the interval of the paired difference, $[0.059, 0.198]$, excludes zero. Of the 15 incidents on which the two methods disagree at top-1, the multimodal method is correct on 14 (exact McNemar $p = 0.001$). At AC@5 the gain is smaller (0.416 to 0.475; 13 versus 7 discordant cases, $p = 0.26$), and its difference interval, $[-0.030, 0.149]$, includes zero. Fusion therefore mainly reorders candidates that metrics alone already place near the top. Fault-type accuracy given a correct top-1 service also rises from 0.222 to 0.645, because the fused method attributes the fault to the signal on which the service ranks highest. H1 is supported.

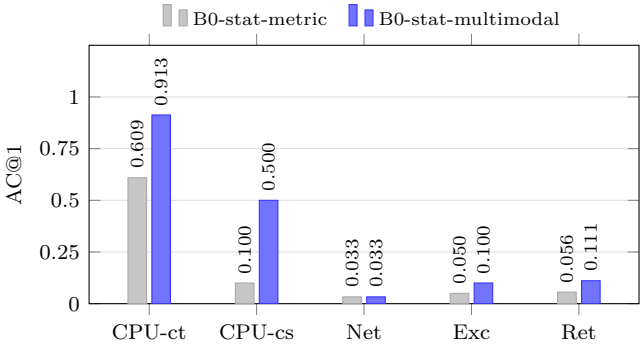


Fig. 2. AC@1 by fault type for the two statistical baselines (case counts: CPU-ct 23, CPU-cs 10, Net 30, Exc 20, Ret 18). Fusion helps resource faults but leaves network and code-level faults largely unresolved.

Fig. 2 breaks AC@1 down by fault type. Resource faults are localized well (CPU contention 0.913, CPU consumed 0.500), but code-level and network faults remain poorly localized (exception 0.100, wrong return value 0.111, network delay 0.033). Network delay is the largest class yet benefits least from fusion. Performance also differs sharply by system: OB reaches AC@1 0.429 and AC@5 0.732, whereas on TT AC@1, AC@3, and AC@5 are all 0.156, and all seven correct TT localizations are CPU-contention faults.

B. G1: Propagation errors (RQ2)

B0-stat-multimodal makes 70 top-1 errors. In 25 of them (35.7%), the top-ranked service is a direct caller or callee of the true root cause (category F1, Table VII), which is the signature of a symptom ranked above its cause. F1 dominates network-delay errors (15 of 29), consistent with delay propagating to adjacent services. H2 is supported: more than one third of errors are, in principle, correctable by a verifier that examines the immediate neighbourhood of the top hypothesis. The verifier itself has not yet been evaluated, so G1 remains open.

C. G2: Cost (RQ3)

Table V reports the cost recorded by the harness. Both statistical baselines run in milliseconds per case without model or tool calls. For the LLM-based methods, the scripted provider demonstrates that the harness counts calls for every case: B1 issues exactly one model call, and P0 issues three model calls and five tool calls per case under the configured caps of 8 steps and 20 tool calls.

TABLE IV
GAP-RESOLUTION SUMMARY: EACH RESEARCH GAP, THE COMPONENT THAT ADDRESSES IT, ITS IMPLEMENTATION STATUS, AND THE EVIDENCE OBTAINED IN THIS PAPER.

Gap	**Limitation in prior work**	**Component**	**Status**	**Evidence in this paper**
G4: multimodal grounding	Single-modality scoring misses log- and trace-visible faults	Six telemetry tools; RRF fusion in B0-stat-multimodal	Implemented	AC@1 0.178 → 0.307 ($p = 0.001$); gain mainly from CPU and memory fusion (Table VI). **Partially resolved.**
G1: hypothesis verification	Agents accept the first plausible hypothesis	Verifier hook in P0	Interface only	25 of 70 top-1 errors rank a call-graph neighbour first (Table VII); quantifies the target. **Open.**
G2: cost awareness	Cost of agentic RCA unbounded and unreported	Step/tool budgets; `Usage` accounting in harness	Caps implemented; adaptive policy planned	Cost recorded per case for all methods (Table V). **Partially resolved.**
G3: incident memory	Every incident investigated from scratch	`Memory` protocol (recall/store)	Interface only	No result yet. **Open.**

TABLE V
MEAN PER-INCIDENT COST RECORDED BY THE HARNESS. ROWS MARKED † USE THE SCRIPTED PROVIDER AND VALIDATE ACCOUNTING ONLY.

Method	**LLM calls**	**Tool calls**	**Tokens**	**Latency (s)**
B0-stat-metric	0	0	0	0.028
B0-stat-multimodal	0	0	0	0.009
B1-single-call†	1	0	–	<0.001
P0-react-agent†	3	5	–	0.028

These counts reflect the fixed script, not the behaviour of a real LLM, and the scripted provider reports no token usage. H3 is therefore verified at the pipeline level: investigation is bounded and its cost is accounted for, while the cost–accuracy trade-off for real models remains to be measured.

D. G3: Memory and RQ4

The memory protocol is implemented as an interface, but no memory policy has been evaluated, and real-LLM runs of B1 and P0 have not yet been performed. We therefore make no claim about whether the agent improves on B0. As an integration check, the scripted B1 and P0 runs reproduce the AC@1 of B0-stat-metric (0.178) exactly, which is expected because the script answers with the metric-anomaly ranking; it confirms that tool outputs, parsing, and scoring are consistent end to end.

VIII. ABLATION AND COMPARATIVE ANALYSIS

Table VI isolates the contribution of each fused signal. Removing CPU or memory evidence lowers AC@1 by 0.069 and 0.059, respectively, and removing CPU reduces TT AC@1 from 0.156 to 0.067. Removing server latency or trace latency lowers AC@1 by only 0.010, and removing log evidence slightly *increases* AC@1 to 0.317 while lowering AC@3 and AC@5. A fusion of the three metric signals alone (CPU, memory, server latency) attains AC@1 0.297 and AC@5 0.485, within 0.010 of the full five-signal fusion.

TABLE VI
SIGNAL ABLATION OF B0-STAT-MULTIMODAL (RRF OVER FIVE SIGNALS). "−" REMOVES ONE SIGNAL; "ONLY" USES ONE SIGNAL.

Configuration	**AC@1**	**AC@3**	**AC@5**	**Avg@5**	**OB**	**TT**
Full fusion (5 signals)	0.307	0.396	0.475	0.386	0.429	0.156
− CPU	0.238	0.366	0.446	0.358	0.375	0.067
− memory	0.248	0.366	0.436	0.360	0.339	0.133
− server p90 latency	0.297	0.416	0.475	0.398	0.411	0.156
− trace latency	0.297	0.396	0.475	0.386	0.411	0.156
− log	0.317	0.376	0.446	0.382	0.446	0.156
CPU only	0.287	0.356	0.436	0.360	0.393	0.156
memory only	0.277	0.485	0.594	0.450	0.446	0.067
server p90 only	0.178	0.337	0.455	0.321	0.304	0.022
trace only	0.218	0.327	0.465	0.341	0.321	0.089
log only	0.059	0.198	0.317	0.192	0.107	0.000
Metric signals only (3)	0.297	0.396	0.485	0.392	0.411	0.156

Thus, under the current scoring functions, most of the improvement over B0-stat-metric arises from *rank-level fusion of complementary resource metrics*, rather than from logs or traces. This qualifies the conclusion of RQ1: multimodal data are available to the methods, but the log and trace scores used by B0 are too coarse to localize the exception, return, and network faults where they would matter most. This observation motivates on-demand inspection of logs and traces by the agent, rather than fixed aggregate scores.

Single-signal rankings are informative as well. Memory alone yields the highest AC@3 (0.485) and AC@5 (0.594) of any configuration but a lower AC@1 (0.277), indicating that memory deviation is a good candidate filter but a poor discriminator at the top. Log evidence alone is weak (AC@1 0.059), and its TT AC@1 is zero.

Efficiency trade-offs. The statistical baselines cost no model calls and run in roughly 0.01–0.03 s per incident, so their accuracy defines the bar that an agent must clear to justify its cost. Each P0 step adds one model call, and the configured caps bound an investigation to at most nine model calls (eight steps plus a forced final

TABLE VII
OUTCOME CATEGORIES OF B0-STAT-MULTIMODAL (COUNTS OF INCIDENTS). F1: TOP-1 IS A DIRECT NEIGHBOUR OF THE ROOT CAUSE; F2: ROOT CAUSE IN TOP-5 OTHERWISE; F3: ROOT CAUSE RANKED BELOW FIFTH.

Group	Hit@1	F1	F2	F3	Total
Online Boutique	24	13	10	9	56
Train-Ticket	7	12	0	26	45
CPU contention	21	2	0	0	23
CPU consumed	5	4	1	0	10
Network delay	1	15	6	8	30
Exception	2	2	2	14	20
Wrong return	2	2	1	13	18
All	**31**	**25**	**10**	**35**	**101**

answer) and twenty tool calls. Because tool observations are truncated to 4000 characters, the prompt grows by at most a bounded amount per tool call; the actual token cost depends on the model and is recorded by the harness when the provider reports it. B1 represents the lowest-cost LLM configuration, with one call over precomputed summaries, and isolates the value of iterative tool use in P0 from the value of LLM reasoning alone.

Comparison with the base paper. OpenRCA grants its agent unrestricted code execution over 68 GB of telemetry and reports an 11.34% solve rate for its best configuration [15]. MicroRCA-Agent instead pre-aggregates telemetry into small bundles and exposes six bounded tools, trading flexibility for auditability and predictable cost. Whether this restricted action space improves accuracy relative to code execution is an empirical question that the planned OpenRCA re-run and the B1/P0 comparison are designed to answer.

IX. Failure Analysis

Table VII gives the frequency of each failure category for B0-stat-multimodal, by system and by fault type.

F3, missed root causes (35 incidents, 50% of errors). This is the largest category and is concentrated in code-level faults (27 of the 38 exception and wrong-return cases) and in TT (26 of 45 cases). The median rank of the true root cause is 2 in OB but 20 in TT, where 46 services dilute the signal. Exceptions and wrong return values perturb application behaviour without large resource deviations, so CPU- and memory-dominated fusion ranks them low. A representative case is `2022-08-22_04_00`, a wrong-return fault in OB's `frontend`, where `paymentservice` ranks first on a CPU deviation and the true cause ranks seventh.

F1, propagation (25 incidents, 36% of errors). The top-ranked service is adjacent to the root cause. In `2022-08-22_04_02`, an exception injected into OB's `frontend` causes its callee `recommendationservice` to rank first on server p90 latency, while `frontend` ranks seventh. In `2022-08-22_05_02`, CPU contention in `currencyservice` is outranked by its caller `frontend`, with the true cause third.

F2, near misses (10 incidents, 14% of errors). All occur in OB, where the small service set keeps the root cause within the top five. In `2022-08-22_05_00`, an exception in `checkoutservice` is ranked second behind `recommendationservice`.

Systematic attractors. Misranking is not uniformly distributed across services: `recommendationservice` is the false top-1 in 18 of 32 OB errors, and `ts-order-service` in 14 of 38 TT errors. Services with naturally volatile latency or high fan-in thus act as attractors for anomaly-based scoring. Two implications follow for the agent: a verifier should test neighbouring services before accepting a hypothesis (F1), and investigation of logs and traces should be directed by the call graph rather than by aggregate anomaly scores (F3).

X. Limitations and Threats to Validity

Internal validity. The central limitation is that no real-LLM results for B1 or P0 are reported; the scripted provider validates integration only and cannot support any claim about agent quality. The B0 scoring constants (e.g., the 0.3 weight on new log templates, the scale floors) were chosen during development on the same 101 incidents, so mild overfitting cannot be excluded. Services with identical fused scores are ordered by their position in the service list, which can affect AC@k on ties. Latency measurements are wall-clock times on a single machine and vary by a few milliseconds between runs.

Construct validity. AC@k treats the labelled service as the sole root cause; incidents in which several services are legitimately implicated are scored as failures. Fault-type accuracy is conditional on a correct top-1 service and is therefore computed on 18–31 incidents, which makes it unstable. Our failure categories use only one-hop call-graph adjacency and may underestimate multi-hop propagation.

External validity. The evaluation covers 101 incidents from two open-source benchmark systems with injected faults. Real production incidents involve larger topologies, concurrent faults, configuration changes, and noisier telemetry [2], [3]. The small per-category counts (as few as 7 cases per fault type in some systems) yield wide confidence intervals, and the TT results show that conclusions drawn on OB do not transfer to larger systems. Results with LLMs will additionally depend on the choice of model, prompt, and provider, and on possible exposure of public benchmark data during model pretraining.

Reproducibility. Preprocessing discards most raw telemetry; methods that require full-resolution logs or traces cannot be evaluated on the bundles without re-running the pipeline.

XI. Conclusion

This paper presented MicroRCA-Agent, a framework that places statistical, single-call, and tool-using agentic RCA methods behind one contract, one tool layer, and one evaluation harness that records both accuracy and cost. On 101 Nezha incidents, rank fusion of multimodal evidence significantly improves top-1 localization over metrics-only scoring (AC@1 0.307 versus 0.178), but an ablation shows that this gain derives mostly from fusing resource metrics, and a failure analysis shows that more than one third of the remaining errors rank a call-graph neighbour of the root cause first, while code-level faults and the larger Train-Ticket system remain largely unresolved. These findings define concrete targets for agentic investigation.

Future work will (i) run B1 and P0 with the same model on all 101 incidents and report accuracy with cost; (ii) implement and evaluate a neighbourhood-checking verifier for G1, an adaptive budget policy for G2, and an incident memory for G3, each through ablation; (iii) re-run OpenRCA's agent under our protocol; and (iv) evaluate on RCAEval as a held-out benchmark [21].

References

[1] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*. Springer, 2017, pp. 195–216. [Online]. Available: https://doi.org/10.1007/978-3-319-67425-4_12

[2] J. Soldani and A. Brogi, "Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: A survey," *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–39, 2022. [Online]. Available: https://doi.org/10.1145/3501297

[3] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, and D. Ding, "Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study," *IEEE Trans. Software Engineering*, vol. 47, no. 2, pp. 243–260, 2021. [Online]. Available: https://doi.org/10.1109/TSE.2018.2887384

[4] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "MicroRCA: Root cause localization of performance issues in microservices," in *Proc. IEEE/IFIP Network Operations and Management Symp. (NOMS)*, 2020, pp. 1–9. [Online]. Available: https://doi.org/10.1109/NOMS47738.2020.9110353

[5] P. Wang, J. Xu, M. Ma, W. Lin, D. Pan, Y. Wang, and P. Chen, "CloudRanger: Root cause identification for cloud native systems," in *Proc. 18th IEEE/ACM Int. Symp. on Cluster, Cloud and Grid Computing (CCGRID)*, 2018, pp. 492–502. [Online]. Available: https://doi.org/10.1109/CCGRID.2018.00076

[6] Y. Meng, S. Zhang, Y. Sun, R. Zhang, Z. Hu, Y. Zhang, C. Jia, Z. Wang, and D. Pei, "Localizing failure root causes in a microservice through causality inference," in *Proc. IEEE/ACM 28th Int. Symp. on Quality of Service (IWQoS)*, 2020, pp. 1–10. [Online]. Available: https://doi.org/10.1109/IWQoS49365.2020.9213058

[7] G. Yu, P. Chen, H. Chen, Z. Guan, Z. Huang, L. Jing, T. Weng, X. Sun, and X. Li, "MicroRank: End-to-end latency issue localization with extended spectrum analysis in microservice environments," in *Proc. Web Conference (WWW)*, 2021, pp. 3087–3098. [Online]. Available: https://doi.org/10.1145/3442381.3449905

[8] Z. Li, J. Chen, R. Jiao, N. Zhao, Z. Wang, S. Zhang, Y. Wu, L. Jiang, L. Yan, Z. Wang, Z. Chen, W. Zhang, X. Nie, K. Sui, and D. Pei, "Practical root cause localization for microservice systems via trace analysis," in *Proc. IEEE/ACM 29th Int. Symp. on Quality of Service (IWQoS)*, 2021, pp. 1–10. [Online]. Available: https://doi.org/10.1109/IWQOS52092.2021.9521340

[9] M. Li, Z. Li, K. Yin, X. Nie, W. Zhang, K. Sui, and D. Pei, "Causal inference-based root cause analysis for online service systems with intervention recognition," in *Proc. 28th ACM SIGKDD Conf. on Knowledge Discovery and Data Mining (KDD)*, 2022, pp. 3230–3240. [Online]. Available: https://doi.org/10.1145/3534678.3539041

[10] L. Pham, H. Ha, and H. Zhang, "BARO: Robust root cause analysis for microservices via multivariate Bayesian online change point detection," *Proc. ACM on Software Engineering*, vol. 1, no. FSE, pp. 2214–2237, 2024. [Online]. Available: https://doi.org/10.1145/3660805

[11] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, and S. Rajmohan, "Recommending root-cause and mitigation steps for cloud incidents using large language models," in *Proc. IEEE/ACM 45th Int. Conf. on Software Engineering (ICSE)*, 2023, pp. 1737–1749. [Online]. Available: https://doi.org/10.1109/ICSE48619.2023.00149

[12] Y. Chen, H. Xie, M. Ma, Y. Kang, X. Gao, L. Shi, Y. Cao, X. Gao, H. Fan, M. Wen, J. Zeng, S. Ghosh, X. Zhang, C. Zhang, Q. Lin, S. Rajmohan, D. Zhang, and T. Xu, "Automatic root cause analysis via large language models for cloud incidents," in *Proc. 19th European Conf. on Computer Systems (EuroSys)*, 2024, pp. 674–688. [Online]. Available: https://doi.org/10.1145/3627703.3629553

[13] Z. Wang, Z. Liu, Y. Zhang, A. Zhong, J. Wang, F. Yin, L. Fan, L. Wu, and Q. Wen, "RCAgent: Cloud root cause analysis by autonomous agents with tool-augmented large language models," in *Proc. 33rd ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 2024, pp. 4966–4974. [Online]. Available: https://doi.org/10.1145/3627673.3680016

[14] D. Roy, X. Zhang, R. Bhave, C. Bansal, P. Las-Casas, R. Fonseca, and S. Rajmohan, "Exploring LLM-based agents for root cause analysis," in *Companion Proc. 32nd ACM Int. Conf. on the Foundations of Software Engineering (FSE Companion)*, 2024, pp. 208–219. [Online]. Available: https://doi.org/10.1145/3663529.3663841

[15] J. Xu, Q. Zhang, Z. Zhong, S. He, C. Zhang, Q. Lin, D. Pei, P. He, D. Zhang, and Q. Zhang, "OpenRCA: Can large language models locate the root cause of software failures?" in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2025. [Online]. Available: https://openreview.net/forum?id=M4qNIzQYpd

[16] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," 2023, arXiv:2305.05176. [Online]. Available: https://arxiv.org/abs/2305.05176

[17] G. Yu, P. Chen, Y. Li, H. Chen, X. Li, and Z. Zheng, "Nezha: Interpretable fine-grained root causes analysis for microservices on multi-modal observability data," in *Proc. 31st ACM Joint European Software Engineering Conf. and Symp. on the Foundations of Software Engineering (ESEC/FSE)*, 2023, pp. 553–565. [Online]. Available: https://doi.org/10.1145/3611643.3616249

[18] C. Lee, T. Yang, Z. Chen, Y. Su, and M. R. Lyu, "Eadro: An end-to-end troubleshooting framework for microservices on multi-source data," in *Proc. IEEE/ACM 45th Int. Conf. on Software Engineering (ICSE)*, 2023, arXiv:2302.05092. [Online]. Available: https://arxiv.org/abs/2302.05092

[19] AI4015 Group G2-07, "MicroRCA-Agent: Agentic root cause analysis for microservice systems," GitHub repository, 2026. [Online]. Available: https://github.com/Ibrahim-Umar-Malik/Agentic_Project

[20] H. Wang, Z. Wu, H. Jiang, Y. Huang, J. Wang, S. Kopru, and T. Xie, "Groot: An event-graph-based approach for root cause analysis in industrial settings," in *Proc. 36th IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2021, pp. 419–429. [Online]. Available: https://doi.org/10.1109/ASE51524.2021.9678708

[21] L. Pham, H. Zhang, H. Ha, F. Salim, and X. Zhang, "RCAEval: A benchmark for root cause analysis of microservice systems with telemetry data," in *Companion Proc. ACM Web Conference (WWW Companion)*, 2025, pp. 777–780. [Online]. Available: https://doi.org/10.1145/3701716.3715290

[22] Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rathi, N. Katarki, A. Bruno, J. Hu, B. Ritchken, B. Jackson *et al.*, "An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems," in *Proc. 24th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 3–18. [Online]. Available: https://doi.org/10.1145/3297858.3304013

[23] Google Cloud Platform, "Online boutique: A cloud-first microservices demo application," GitHub repository, 2024. [Online]. Available: https://github.com/GoogleCloudPlatform/microservices-demo

[24] C. Leys, C. Ley, O. Klein, P. Bernard, and L. Licata, "Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median," *Journal of Experimental Social Psychology*, vol. 49, no. 4, pp. 764–766, 2013. [Online]. Available: https://doi.org/10.1016/j.jesp.2013.03.013

[25] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. on Research and Development in Information Retrieval*, 2009, pp. 758–759. [Online]. Available: https://doi.org/10.1145/1571941.1572114

[26] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, arXiv:2201.11903. [Online]. Available: https://arxiv.org/abs/2201.11903

[27] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2023, arXiv:2210.03629. [Online]. Available: https://arxiv.org/abs/2210.03629

[28] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," 2023, arXiv:2302.04761. [Online]. Available: https://arxiv.org/abs/2302.04761

[29] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2023, arXiv:2203.11171. [Online]. Available: https://arxiv.org/abs/2203.11171

[30] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," 2023, arXiv:2303.17651. [Online]. Available: https://arxiv.org/abs/2303.17651

[31] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," 2023, arXiv:2303.11366. [Online]. Available: https://arxiv.org/abs/2303.11366

[32] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. 36th Annual ACM Symp. on User Interface Software and Technology (UIST)*, 2023, pp. 1–22. [Online]. Available: https://doi.org/10.1145/3586183.3606763

[33] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, 2024. [Online]. Available: https://doi.org/10.1007/s11704-024-40231-1

[34] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. on Web Services (ICWS)*, 2017, pp. 33–40. [Online]. Available: https://doi.org/10.1109/ICWS.2017.13